



Institut Supérieur d'Informatique et de Mathématiques de Monastir

ISIMM

Devoir Surveillé – S2 – 2024/2025

Filière : 1 LInfo	Matière : Système exploitation 2		Enseignant : Sana BENZARTI
Date : 17/ 03 / 2025	Nbr de Crédits : 4	Coefficient : 2	Documents autorisés : Non Calculatrice autorisée : oui
Durée de l'examen : 1h	Régime d'évaluation : Mixte / CC		Nombre de pages : 2
	Ex(70%) + DS(10%) + TP(20%)		

La clarté et la propreté de la feuille de l'examen sont prises en compte

Exercice 1 : QCM: Vous pouvez trouver plusieurs réponses exactes pour une même question

1- Les mises à jour sont proposées à un rythme frénétique, il s'agit de : ☒ a- Windows b- Linux c- Mac OS	6- La fonction waitpid() renvoie : ☒ a- 0 chez le processus père ☒ b- Le PID du fils chez le père ☒ c- -1 en cas d'erreur
2- Les serveurs utilisent des processeurs de type : ☒ a- Intel XEON Platinum ☒ b- Intel XEON GOLD c- Intel Core i3	7- La macro WTERMSIG : ☒ a- Peut être utilisée avec wait(&status) ☒ b- Renvoie le numéro d'un signal spécifique c- Renvoie le code de sortie en cas de fin de tâche avec succès
3- La famille exec comporte : ☒ a- execlp ☒ b- execve c- execl	8- La syntaxe exacte d'execl est : a- execl("find", "/home/rep", "-type", "f", NULL); ☒ b- execl("/bin/find", "find", "/home/rep", "-type", "f", NULL); c- execl("find", "/home/rep", "-type", "f", NULL, NULL);
4- Si un processus fils meurt avant le processus père sans le notifier, il devient : ☒ a- Orphelin ☒ b- Zombie c- Zombie et orphelin	9- L'appel système exec fait partie de la bibliothèque : a- sys/wait.h ☒ b- unistd.h ☒ c- stdlib.h
5- Les processus orphelins sont adoptés par le processus : a- Père ☒ b- Init ☒ c- Systemd	10- La fonction exit(EXIT_FAILURE) : ☒ a- Termine le processus immédiatement avec erreur b- Equivalent a exit(-1)

Exercice 2 : Les fonctions fork(), wait() et exit()

A- On considère les codes 1 et 2. Mentionner le nombre de processus générés en expliquant votre démarche.

Code 1	Code 2
<pre>int main() { int cpt=0; while (cpt < 3) { if (fork() > 0) cpt++; else cpt=3; } }</pre>	<pre>int main() { int i; for (i=0; i<2; i++) fork(); printf("hello!\n"); exit(0); }</pre>

- B- En considérant le code 4, mentionner le résultat trouvé avec l'état du processus fils.
C- Modifier le code pour avoir une bonne synchronisation entre le processus père et le processus fils (ne pas réécrire le code, indiquer ce que vous avez ajouté)
D- Ajouter au processus fils la commande `execvp(argv[1], &argv[1])` et au processus père les macros `WIFEXITED(status)` et `WEXITSTATUS(status)`.

Code 4
<pre>pid_t val; val = fork(); if (val == 0) { printf("Fils : Je suis le fils, la valeur de retour du fork dans mon contexte est %d.\n", val); printf("Fils : Termine !\n"); } else if (val > 0) { printf("Père : Je suis le père, la valeur de retour du fork dans mon contexte est %d.\n", val); while (1) // Boucle infinie, le père ne termine jamais ! usleep(1); }</pre>

Exercice 3 : Ordonnancement des processus avec commutation de contexte CC

Soit le tableau suivant : Si deux processus arrivent au même temps, on les place dans la FA par ordre alphanumérique.

Processus	Temps d'exécution	Temps d'arrivée
P1	70	0
P2	40	0
P3	60	10
P4	10	10
P5	20	15
P6	40	20
P7	10	20

En utilisant l'algorithme d'ordonnancement Round Robin (Quantum = 30, TCC=10 et file d'attente en FIFO).

- Donner le diagramme de Gantt
- Calculer sous forme de tableau : le temps de rotation, le temps d'attente, la moyenne du temps de rotation, la moyenne du temps d'attente et le débit (mettre les détails du calculs).

T.R.M

T.A.M

debut

Bon travail